VibeShield

Deterministic vulnerability research • Academic capstone documentation

Version 1.0 | 6 October 2026

1. Purpose and scope

VibeShield brings static repository analysis and local website checks into one explainable workflow. It is designed for cybersecurity coursework: the researcher submits an authorized target, inspects candidate findings with their source evidence, and exports a traceable report. The project uses procedural Python, dataclasses, subprocess wrappers, Python AST inspection, and a React interface. No model chooses a vulnerability type, confidence, severity, or remediation.

A candidate is an observation requiring review. It is not proof of exploitation. An empty result is not proof of security, particularly when scanner stages fail. These distinctions appear in the interface, command-line output, and reports.

Website scanning is restricted to localhost, 127.0.0.1, and ::1. Public GitHub repositories and local directories can be statically analyzed after the user confirms authorization. All project testing targets the deliberately vulnerable local fixture. Network sandbox isolation, authenticated website workflows, exploit verification, and numeric risk scoring are future phases.

2. Architecture and data flow

The browser sends a small JSON request to the loopback Python HTTP server. The server validates the target and authorization, creates a persistent job, and queues scanner stages in a single background worker. The browser polls status and scan records every two seconds while visible. Completed stages append actual findings; failures remain explicit in the record.

Static path: local directory or shallow public GitHub checkout → Semgrep → AST heuristic → Gitleaks → consolidated candidate list.

Dynamic path: validated loopback URL → Nuclei bundled header template → ZAP passive analysis → consolidated candidate list.

Report path: stored job plus findings, stage outcomes, and errors → deterministic review guidance → PDF or JSON download. Reports are available after completed, partial, failed, or interrupted runs. An incomplete report preserves the evidence rather than silently treating missing coverage as clean.

The dashboard is a local single-user application. The Python server serves the compiled React assets and API from the same origin. It binds to 127.0.0.1. No hosted database, account system, cloud inference service, or telemetry integration is added by the dashboard.

3. Repository guide

run_scan.py is the root command-line entry point. run_dashboard.py starts the local dashboard. pipeline/scan/common.py supplies shared findings, subprocess execution, errors, repository validation, and URL validation. Individual runner modules translate external formats into shared findings. pipeline/scan/run_scan.py consolidates the results.

pipeline/scan/ast_heuristic_analyzer.py implements the Flask ownership heuristic. pipeline/scan/rules/flask.yaml contains a custom Semgrep rule. pipeline/scan/templates/missing-nosniff.yaml contains the narrow Nuclei template. Tests under pipeline/scan/tests cover detection and runner behavior.

dashboard/server.py implements validation, job persistence, scheduling, API routes, and frontend serving. dashboard/web/src/main.jsx contains the React views and interactions. style.css defines the responsive visual system. pipeline/report/report_builder.py implements JSON/PDF reports. dashboard/tests exercises backend behavior. dashboard/web/tests contains the browser workflow test.

fixtures/idor-notes-app/app.py is the deliberately vulnerable fixture. Its original contents are preserved. The earlier described idor_prober.py is absent from this checkout and has not been recreated; the dashboard does not claim to perform its exploit confirmation workflow.

4. Static analysis: Semgrep

semgrep_runner.py invokes Semgrep as a subprocess using the registry auto configuration plus the bundled custom rules. It parses JSON results into RawFinding records: file, line, rule_id, message, severity, source_snippet, and source. Missing tools, timeouts, tool errors, and malformed output produce explicit ScanError failures.

Semgrep detects rule-defined code patterns. The fixture's missing ownership check is a logic issue and is handled by the separate AST heuristic. A configuration observation such as app.run(host='0.0.0.0') needs deployment context: a container-internal wildcard bind does not prove public exposure when host publishing is loopback-only.

The auto configuration requires registry network access. Rule selection and installed tool versions can change results. For reproducible academic experiments, record the scanner version, repository commit, configuration, and date; archive or pin the exact external rule set separately. Absence of AI calls does not mean absence of all network calls.

5. AST ownership heuristic

The built-in Python ast module parses Python files without importing or executing the scanned application. The analyzer recognizes common Flask route decorators, including app and blueprint route/get patterns. Its scope is deliberately narrower than framework-independent authorization analysis.

IDOR-AST-01 looks for an ID-like route parameter reaching a recognized database lookup, with no recognized ownership guard protecting the result. Common lookup forms include execute, query, get, filter, and filter_by. The rule tracks supported local assignments and authenticated-principal shapes to identify relevant comparisons rather than treating every comparison as authorization.

Recognized guards include supported rejecting inline checks before the lookup, and supported ownership checks of a fetched object before disclosure. A comparison used only for logging does not count. Unrelated identity comparisons, checks after a return, and nested helper definitions do not establish protection. Tests cover these distinctions.

The fixture's get_note() contains the route identifier note_id and uses it in the SQL lookup at app.py:122. Authentication exists, but the function does not enforce resource ownership before returning the note. The analyzer therefore emits an IDOR candidate with MEDIUM confidence and a plain-language reason. Confidence describes the heuristic evidence; no severity is assigned to this finding.

Clean test cases constrain or reject access with a supported ownership check. They are not flagged. This demonstrates that the rule checks relationships between the route identifier, lookup, principal, and control flow, rather than flagging every route handler.

Limitations: decorators, middleware, helper functions, service layers, custom ORM wrappers, complex aliases, dynamic attribute access, and policies outside the function may be missed. These can create false positives or false negatives. The analyzer is not a complete control-flow engine or proof of authorization. Syntax errors are reported as failures, not silently interpreted as clean scans. See the analyzer module docstring and tests for exact supported shapes.

6. Secret scanning

secrets_runner.py invokes Gitleaks and parses its JSON report. Secret values are redacted before findings are consolidated or displayed. A secret-pattern candidate does not establish that a credential is active. Review suspected credentials locally, revoke or rotate genuine exposures, and avoid placing live values in shared reports.

The fixture produced no Gitleaks candidates in the recorded scanner verification. A missing report, missing executable, timeout, or malformed result is an error rather than a clean scan. Repository-history coverage depends on the chosen scanner invocation; the dashboard's public GitHub checkout is shallow and should not be described as complete historical coverage.

7. Dynamic scanning

nuclei_runner.py invokes Nuclei and parses newline-delimited JSON into the shared finding shape. The current implementation uses one bundled missing X-Content-Type-Options: nosniff template. It does not claim the coverage of the full Nuclei community template collection. No AI template generation is used.

zap_runner.py supports a compatible baseline executable, native ZAP automation, and a Docker fallback. Native automation is configured for passive analysis. ZAP findings preserve their source and affected endpoint. Nuclei and ZAP can both identify the same missing header; overlap is deliberately retained so source evidence is not lost.

Both wrappers validate loopback targets. The fixture has no root page, so the appropriate smoke-test URL is http://127.0.0.1:5000/api/health. Native ZAP requires a compatible Java runtime. Docker fallback requires an accessible daemon and image; finding a Docker executable alone does not prove readiness.

Input validation is not full network isolation. Scanner internals, redirects, external rule downloads, and tool update behavior are not confined by a sandbox in this phase. Use only the controlled fixture and do not treat the local target allowlist as an egress firewall.

8. Dashboard workflow

Overview provides three target choices: Website / IP, GitHub repository, and Local project. The target field includes a relevant example and a scope explanation. The ownership/permission checkbox must be selected before submission. Use test fixture selects the local fixture directory without automatically granting consent.

The coverage panel shows which tools apply and whether executables can be found. A discovered executable can still fail at runtime; the scan record is the authority for actual coverage. Progress lists real queued, running, completed, and failed stages without inventing a percentage.

Scans preserves history and supports target/ID search and status filtering. Findings supports scan selection, severity filtering, and evidence search. Opening a candidate displays its affected location, observation, source evidence when available, rule identifier, confidence, and candidate status.

Reports offers PDF and JSON exports for terminal runs. Settings explains tool readiness and environment-variable configuration. The interface adapts to a compact navigation menu on mobile, preserves keyboard focus indicators, and respects reduced-motion preferences. Manrope fonts and SVG icons are served locally with the application bundle.

9. Backend, persistence, and API

POST /api/scans accepts target_type, target, and authorized. The request requires the session token returned by GET /api/status. Host and Origin checks limit access to the dashboard's localhost origin. Invalid JSON, oversized requests, unsupported targets, and missing authorization receive explicit errors.

GET /api/status returns tool availability, the session token, local scope, fixture shortcuts, Git availability, and PDF availability. GET /api/scans lists stored records. GET /api/scans/{id} returns an individual job. GET /api/scans/{id}/report.pdf and report.json export a finished job. GET /api/documentation.pdf downloads this project document.

Each job stores its identifier, target type, normalized target, authorization declaration, timestamps, status, tool stages, findings, errors, and repository commit where applicable. The job directory defaults to .vibeshield/scans with private directory permissions. Atomic file replacement protects individual JSON updates. On restart, unfinished jobs are marked interrupted rather than silently resumed or reported complete.

One worker processes scans sequentially. At most five unfinished scans may be queued. Scanner stages use bounded timeouts; the shared subprocess helper terminates process groups on timeout. History retention, per-user access control, cancellation, and production resource isolation are not implemented.

GitHub input accepts a public repository URL, not credentials or a branch/tree URL. The server performs a shallow clone with prompts, credential helpers, user Git configuration, and hooks disabled. It records the checked-out commit and removes the temporary checkout afterward. A post-clone 200 MiB limit prevents analysis of oversized checkouts, but does not bound the bytes downloaded during cloning. Repository code and package install scripts are not executed.

10. Finding and report schemas

The CLI emits a JSON list. Each item contains file_or_endpoint, vulnerability_type, confidence, source_tool, and details. Details retain the native rule or heuristic data and CANDIDATE status. Dynamic sources remain distinct; no cross-tool deduplication occurs.

Dashboard reports wrap that candidate list with schema_version, scan_id, target, target_type, timestamps, status, complete, authorization, repository_commit, summary, stages, errors, and limitations. The complete flag is true only when every requested scanner stage completed successfully.

Display severity is derived from native tool severity: WARNING maps to MEDIUM, ERROR to HIGH, INFORMATIONAL to INFO, and absent severity to UNRATED. AST IDOR findings are intentionally unrated. Confidence and severity are separate concepts. No CVSS value or invented numeric risk score is generated.

Review guidance comes from a fixed mapping for known findings such as IDOR, secret exposure, missing nosniff, and version disclosure. Unknown rules receive generic review guidance. PDF content is escaped before being passed to the rendering library. Reports include incomplete coverage and explain that empty results do not establish safety.

11. Installation and operation

Use Python 3.10 or later, Node.js compatible with the installed Vite version, and npm. Create a virtual environment and install dashboard/requirements.txt. Install frontend packages with npm install --prefix dashboard/web, then build with npm run build --prefix dashboard/web. Start with .venv/bin/python run_dashboard.py and open http://127.0.0.1:8787.

External scanners are separate prerequisites. Set SEMGREP_BIN, GITLEAKS_BIN, NUCLEI_BIN, ZAP_BIN, ZAP_CLI_BIN, and DOCKER_BIN as appropriate. Values are executable paths, not shell command strings. Set JAVA_HOME when needed by native ZAP. The AST analyzer needs only Python's standard library. PDF output requires ReportLab, pinned in dashboard/requirements.txt.

CLI examples: python3 run_scan.py --repo-path fixtures/idor-notes-app; python3 run_scan.py --url http://127.0.0.1:5000/api/health. Both flags may be combined. CLI stdout remains a JSON list; errors go to stderr and partial failures return a nonzero exit status.

The pre-existing fixture image can be launched with docker run --rm --name vibeshield-scan-fixture -p 127.0.0.1:5000:5000 vibeshield-idor-notes-app:latest. This checkout does not supply an image build or original compose file. Do not publish the deliberately vulnerable fixture to a public interface.

12. Verification and reproducibility

The scanner suite contains 34 tests and passes in the current checkout. It covers the real fixture, clean ownership patterns, malformed tool output, missing binaries, timeouts, redaction, local scope, preserved overlap, and partial failures. Six dashboard backend tests pass, covering loopback validation, external target rejection, GitHub URL validation, consent, actual AST results in partial reports, persistence, and restart interruption.

Run .venv/bin/python -m unittest discover -s pipeline/scan/tests -v and the equivalent command with -s dashboard/tests. The browser workflow test under dashboard/web/tests runs a real local fixture scan, opens AST evidence, downloads a PDF, checks mobile layout, and asserts no automated accessibility violations on the populated findings view. It requires the dashboard server and a Playwright Chromium installation.

The dashboard browser workflow passed in Chromium with an actual completed static fixture scan and PDF download. Width checks passed at 320, 390, and 768 pixels; desktop and mobile screenshots were reviewed. The populated findings view had zero automated axe violations. See dashboard/VERIFICATION.md for scope and limitations.

Historical scanner smoke-test evidence is recorded in pipeline/scan/VERIFICATION.md. It records two static candidates, three dynamic candidates, and five combined candidates for that fixture and runtime. Those counts are observations, not hardcoded dashboard data. Runtime binaries in temporary locations may no longer exist; current Settings and scan outcomes must be checked independently.

For academic comparison, preserve the exact fixture source, input endpoint, repository commit, tool versions, rule files, configuration, and raw exported report. Timestamps and identifiers vary by run; deterministic detection means the same code and rules make the same detection decisions under the same conditions, not that every report byte is identical.

13. Ethics, limitations, and future work

The project is intended only for systems the researcher owns or has explicit permission to assess. The consent checkbox records a declaration; it is not independent proof of authorization. Local-only website validation implements this phase's scope. GitHub scanning remains static and requires repository authorization.

Limitations include narrow Flask analysis, heuristic false positives and negatives, shallow repository history, one bundled Nuclei check, passive ZAP coverage, no authenticated dynamic session configuration, no deduplication, and dependency on external scanner availability. Browser reports can contain source context and filesystem paths; inspect exported material before sharing it.

Future work may add a separately reviewed exploit-verification integration, resource and network isolation, richer reproducibility manifests, cancellation and retention controls, authenticated fixture workflows, broader tested AST shapes, and a transparent scoring methodology. None of those are claimed as implemented here.

14. Primary references

Python AST: https://docs.python.org/3/library/ast.html Semgrep CLI: https://docs.semgrep.dev/cli-reference Gitleaks: https://github.com/gitleaks/gitleaks Nuclei running guide: https://docs.projectdiscovery.io/opensource/nuclei/running ZAP baseline: https://www.zaproxy.org/docs/docker/baseline-scan/ ZAP automation framework: https://www.zaproxy.org/docs/automate/automation-framework/ OWASP IDOR prevention: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html CWE-639: https://cwe.mitre.org/data/definitions/639.html

Implementation claims in this document describe the source files in this checkout. Tool documentation explains the external mechanisms; the repository tests and verification record provide the project's own evidence.